

Guide Technique - HackathoN Innov UI

Version: 1.0
Date: Janvier 2026
Framework: Angular 21.0.3

Table des matières

- [1. Introduction](#)
- [2. Architecture du projet](#)
- [3. Installation et configuration](#)
- [4. Structure des dossiers](#)
- [5. Composants détaillés](#)
- [6. Services](#)
- [7. Interfaces et types](#)
- [8. Flux de données](#)
- [9. Gestion des états](#)
- [10. Tests](#)
- [11. Déploiement](#)
- [12. Bonnes pratiques](#)
- [13. Dépannage](#)

Introduction

À propos du projet

HackathoN Innov UI est une application web Angular permettant de gérer le cycle complet d'un hackathon interne, de l'inscription des participants jusqu'à la génération automatique d'équipes et le suivi en temps réel de l'événement.

Objectifs

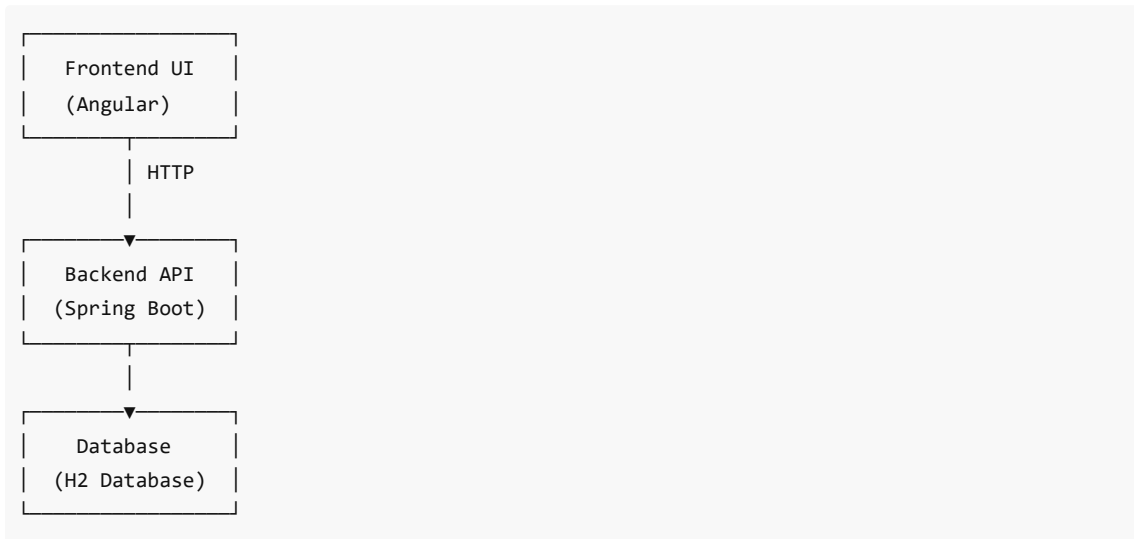
- Automatisation** : Réduire la charge de travail manuel en automatisant les inscriptions et la création d'équipes
- Temps réel** : Suivre l'état du hackathon avec un compte à rebours dynamique
- Expérience utilisateur** : Offrir une interface intuitive pour les participants et le jury
- Scalabilité** : Architecture modulaire permettant l'ajout de nouvelles fonctionnalités

Technologies principales

Technologie	Version	Usage
Angular	21.0.3	Framework frontend
TypeScript	5.9.2	Langage de programmation
RxJS	7.8.0	Programmation réactive
Tailwind CSS	4.1.12	Framework CSS
Vitest	4.0.8	Tests unitaires

Architecture du projet

Architecture globale



Patterns architecturaux

- **Component-based** : Architecture modulaire avec composants réutilisables
- **Service-oriented** : Logique métier encapsulée dans des services
- **Reactive programming** : Utilisation d'Observables RxJS pour la gestion asynchrone
- **Type safety** : Typage strict avec TypeScript et interfaces

Principes SOLID appliqués

- **Single Responsibility** : Chaque composant/service a une responsabilité unique
- **Open/Closed** : Extension sans modification (voir [SOLID_REFACTORING.md](#))
- **Dependency Inversion** : Dépendance sur des abstractions (interfaces)

Installation et configuration

Prérequis

- Node.js >= 20.0.0
- npm >= 11.6.2
- Angular CLI >= 21.0.1

Installation

```
# Cloner le dépôt
git clone [repository-url]
cd hn-innov-ui

# Installer les dépendances
npm install
```

```
# Démarrer le serveur de développement
npm start
```

L'application sera accessible sur `http://localhost:4200/`

Configuration des environnements

Développement (`env.dev.ts`)

```
export const environment = {
  production: false,
  apiUrl: 'http://localhost:8080',
};
```

Production (`env.prd.ts`)

```
export const environment = {
  production: true,
  apiUrl: 'https://hn-innov-api.fly.dev',
};
```

Scripts npm disponibles

Commande	Description
<code>npm start</code>	Démarrer le serveur de développement
<code>npm run build</code>	Build de production
<code>npm test</code>	Exécuter les tests unitaires
<code>npm run watch</code>	Build en mode watch

Structure des dossiers

```
src/
├─ app/
│   ├─ constantes/          # Constantes de configuration
│   │   ├─ dashboard.constants.ts
│   │   └─ index.ts
│   │
│   ├─ env/                 # Configuration d'environnement
│   │   ├─ env.dev.ts
│   │   └─ env.prd.ts
│   │
│   ├─ features/            # Fonctionnalités métier
│   │   ├─ contact-page/    # Page de contact
│   │   ├─ dashboard-page/  # Dashboard principal
│   │   │   ├─ components/  # Sous-composants du dashboard
│   │   │   └─ countdown/
```

```

| | | | | └─ period-card/
| | | | | └─ project-info/
| | | | | └─ stats-card/
| | | | | └─ status-card/
| | | | └─ dashboard-page.ts
| | | | └─ dashboard-page.utils.ts
| | | | └─ dashboard-page.spec.ts
| | | └─ home-page/          # Page d'accueil
| | | └─ register-page/     # Inscription
| | | └─ rules/             # Règles du hackathon
| | | └─ subject-page/      # Gestion des sujets
| |
| └─ header/                # En-tête de navigation
|
| └─ interfaces/            # Interfaces TypeScript
|   └─ countdown.interface.ts
|   └─ period.interface.ts
|   └─ interfaces.ts
|   └─ index.ts
|
| └─ services/              # Services métier
|   └─ countdown.service.ts
|   └─ email.service.ts
|   └─ form-validation.service.ts
|   └─ hackathon.service.ts
|   └─ jury-member.service.ts
|   └─ participant.service.ts
|   └─ payload-builder.service.ts
|   └─ period.service.ts
|   └─ skills.service.ts
|   └─ status-transition.service.ts
|   └─ status.service.ts
|   └─ subject.service.ts
|   └─ team.service.ts
|   └─ index.ts
|
| └─ types/                 # Types TypeScript
|   └─ hackathon-status.type.ts
|
| └─ app.config.ts          # Configuration Angular
| └─ app.routes.ts          # Routes de l'application
| └─ app.ts                 # Composant racine
|
└─ public/                  # Assets statiques
└─ index.html               # Point d'entrée HTML
└─ main.ts                  # Bootstrap Angular
└─ styles.css               # Styles globaux

```

Composants détaillés

Dashboard Page

Fichier: <src/app/features/dashboard-page/dashboard-page.ts>

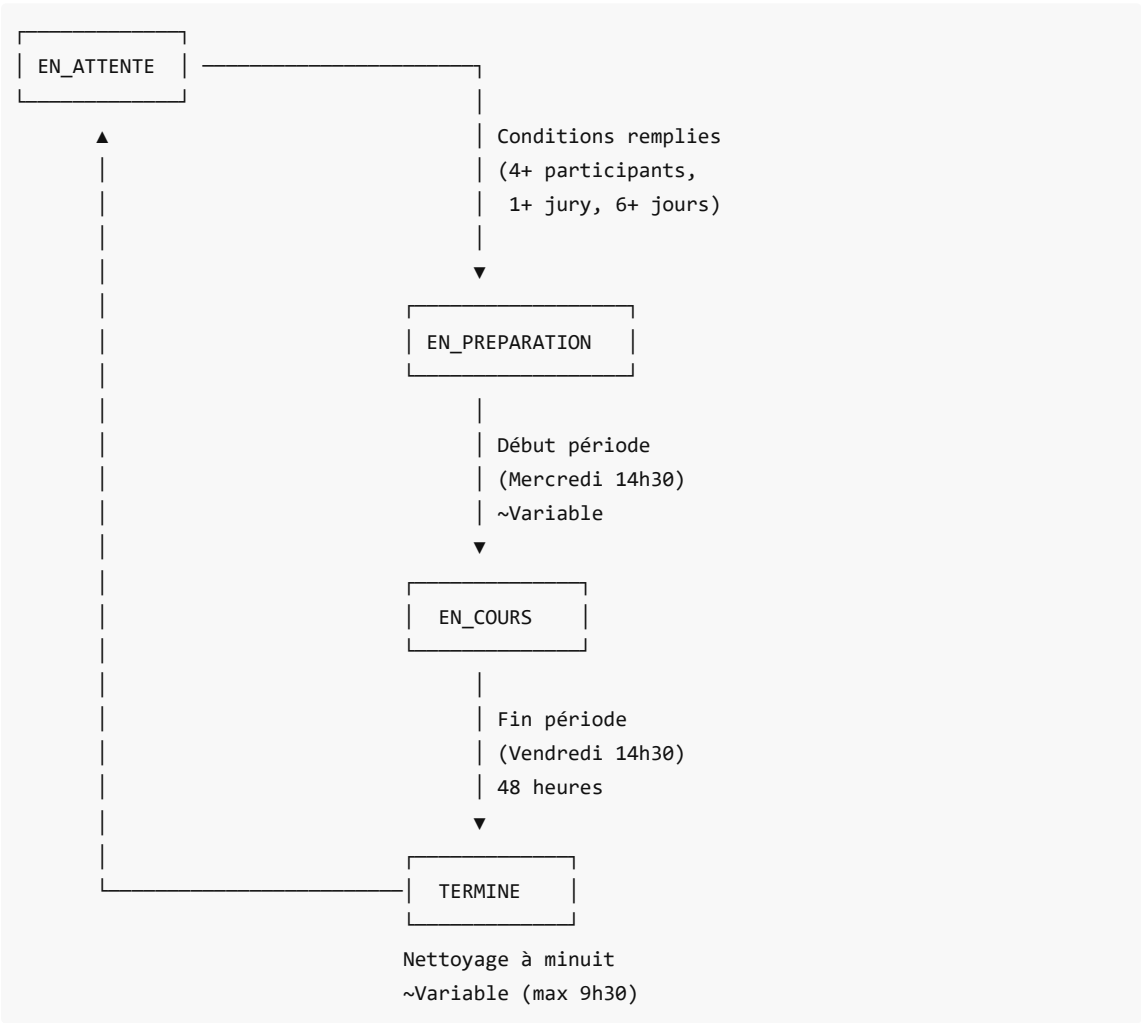
Responsabilité

Composant principal affichant l'état actuel du hackathon et gérant les transitions automatiques entre les différents statuts.

Statuts du hackathon

```
export type HackathonStatus =  
  | 'EN_ATTENTE' // En attente des conditions minimales  
  | 'EN_PREPARATION' // Prêt à démarrer  
  | 'EN_COURS' // Hackathon en cours  
  | 'TERMINE'; // Hackathon terminé
```

Diagramme de transition d'états



Fonctionnalités clés

1. Vérification des conditions

```
checkConditionsAndCreatePeriod(): void {
  // Vérifie si MIN_PARTICIPANTS et MIN_JURY_MEMBERS sont atteints
  // Vérifie le délai MIN_DAYS_BETWEEN_PERIODS
  // Crée automatiquement une période si conditions OK
}
```

2. Génération d'équipes

```
generateTeams(): void {
  // Appelé automatiquement au passage en EN_COURS
  // Crée des équipes équilibrées
  // Sélectionne un jury et son sujet
}
```

3. Nettoyage automatique

```
scheduleCleanup(): void {
  // Planifie le nettoyage pour minuit
  // Supprime toutes les données
  // Remet le statut à EN_ATTENTE
}
```

Sous-composants

CountdownComponent

Affiche un compte à rebours dynamique adapté au statut actuel.

Props:

- `targetDate: Date` - Date cible du compte à rebours
- `status: HackathonStatus` - Statut actuel

StatsCardComponent

Affiche une statistique avec icône et valeur.

Props:

- `icon: string` - Classe d'icône
- `label: string` - Libellé de la statistique
- `value: number` - Valeur à afficher

StatusCardComponent

Affiche le statut actuel avec un badge coloré.

Props:

- `status: HackathonStatus` - Statut à afficher

PeriodCardComponent

Affiche les dates de début et fin du hackathon.

Props:

- `period: Period` - Période du hackathon

ProjectInfoComponent

Affiche les informations sur les équipes et le sujet.

Props:

- `teams: Team[]` - Liste des équipes
- `subject: string` - Sujet du hackathon

Register Page

Fichier: [src/app/features/register-page/register-page.ts](#)

Responsabilité

Gère l'inscription des participants et membres du jury avec validation de formulaire.

Formulaire réactif

```
registrationForm = this.fb.group({
  firstName: ['', Validators.required],
  lastName: ['', Validators.required],
  email: ['', [Validators.required, Validators.email]],
  role: ['participant', Validators.required],
  skill: ['', Validators.required],
  ideaOption: ['adopt', Validators.required],
  subject: [''],
});
```

Compétences disponibles

Compétence	Icône	Description
Développeur		Développement full-stack
Designer		Design UI/UX
Chef de projet		Gestion de projet
Communicant		Communication et marketing

Options d'idée

- **Adopter un sujet existant** : Le participant rejoint un projet proposé par un jury
- **Proposer un nouveau sujet** : Le participant soumet sa propre idée

Validation et soumission

```
onSubmit(): void {
  if (this.registrationForm.valid) {
    // Construction du payload
    const payload = this.buildPayload();

    // Envoi selon le rôle
    if (role === 'participant') {
      this.participantService.register(payload);
    }
  }
}
```

```
    } else {
      this.juryMemberService.register(payload);
    }

    // Email de confirmation
    this.emailService.sendConfirmation(payload.email);
  }
}
```

Home Page

Fichier: [src/app/features/home-page/home-page.ts](#)

Responsabilité

Page d'accueil présentant le concept du hackathon et les actions principales.

Sections

1. **Hero section** : Titre et appel à l'action
 2. **Présentation** : Explication du concept
 3. **Appel à l'action** : Boutons d'inscription et règles
-

Services

HackathonService

Fichier: [src/app/services/hackathon.service.ts](#)

Responsabilité

Gère toutes les opérations liées aux hackathons.

Méthodes principales

```
export class HackathonService {
  // Récupérer tous les hackathons
  getAll(): Observable<Hackathon[]>;

  // Créer un nouveau hackathon
  create(hackathon: Hackathon): Observable<Hackathon>;

  // Générer les équipes
  generateTeams(hackathonId: number): Observable<void>;

  // Récupérer le hackathon actuel
  getCurrent(): Observable<Hackathon | null>;
}
```

StatusService

Fichier: [src/app/services/status.service.ts](#)

Responsabilité

Gère le statut global du hackathon et les transitions d'états.

Méthodes principales

```
export class StatusService {  
  // Récupérer le statut actuel  
  getStatus(): Observable<HackathonStatus>;  
  
  // Mettre à jour le statut  
  updateStatus(status: HackathonStatus): Observable<void>;  
  
  // Observable du statut (pour souscription)  
  status$: Observable<HackathonStatus>;  
}
```

ParticipantService

Fichier: <src/app/services/participant.service.ts>

Responsabilité

Gère les inscriptions et les opérations sur les participants.

Méthodes principales

```
export class ParticipantService {  
  // Inscrire un participant  
  register(participant: Participant): Observable<Participant>;  
  
  // Récupérer tous les participants  
  getAll(): Observable<Participant[]>;  
  
  // Compter les participants  
  count(): Observable<number>;  
  
  // Supprimer tous les participants  
  deleteAll(): Observable<void>;  
}
```

TeamService

Fichier: <src/app/services/team.service.ts>

Responsabilité

Gère les équipes générées pour le hackathon.

Méthodes principales

```
export class TeamService {  
  // Récupérer toutes les équipes  
  getAll(): Observable<Team[]>;  
  
  // Créer une équipe  
  create(team: Team): Observable<Team>;  
  
  // Supprimer toutes les équipes  
  deleteAll(): Observable<void>;  
}
```

EmailService

Fichier: [src/app/services/email.service.ts](#)

Responsabilité

Envoie des notifications par email aux participants.

Méthodes principales

```
export class EmailService {  
  // Envoyer un email de confirmation  
  sendConfirmation(email: string, name: string): Observable<void>;  
  
  // Envoyer un email de rappel  
  sendReminder(email: string, eventDate: Date): Observable<void>;  
}
```

CountdownService

Fichier: [src/app/services/countdown.service.ts](#)

Responsabilité

Calcule et gère les compte à rebours.

Méthodes principales

```
export class CountdownService {  
  // Créer un observable de compte à rebours  
  createCountdown(targetDate: Date): Observable<TimeRemaining>;  
  
  // Formater le temps restant  
  formatTimeRemaining(time: TimeRemaining): string;  
}
```

StatusTransitionService

Fichier: [src/app/services/status-transition.service.ts](#)

Responsabilité

Gère les transitions automatiques entre les différents statuts.

Méthodes principales

```
export class StatusTransitionService {  
  // Vérifier et effectuer les transitions nécessaires  
  checkAndTransition(): Observable<void>;  
  
  // Planifier la prochaine transition  
  scheduleNextTransition(date: Date): void;  
}
```

Interfaces et types

Hackathon

```
export interface Hackathon {  
  id?: number;  
  periodId: number;  
  selectedJuryMemberId: number;  
  createdAt?: Date;  
}
```

Period

```
export interface Period {  
  id?: number;  
  startDate: Date;  
  endDate: Date;  
  createdAt?: Date;  
}
```

Participant

```
export interface Participant {  
  id?: number;  
  firstName: string;  
  lastName: string;  
  email: string;  
  skill: Skill;  
  ideaOption: IdeaOption;  
  subject?: string;  
  createdAt?: Date;  
}
```

JuryMember

```
export interface JuryMember {  
  id?: number;  
  firstName: string;  
  lastName: string;  
  email: string;  
  subject: string;  
  createdAt?: Date;  
}
```

Team

```
export interface Team {  
  id?: number;  
  name: string;  
  members: TeamMember[];  
  hackathonId: number;  
}  
  
export interface TeamMember {  
  id: number;  
  firstName: string;  
  lastName: string;  
  skill: Skill;  
}
```

Types

```
export type HackathonStatus = 'EN_ATTENTE' | 'EN_PREPARATION' | 'EN_COURS' | 'TERMINE';  
  
export type Skill = 'Développeur' | 'Designer' | 'Chef de projet' | 'Communicant';  
  
export type IdeaOption = 'adopt' | 'propose';
```

Flux de données

Cycle complet d'un hackathon

1. INSCRIPTION
 - └ Participant remplit le formulaire
 - └ Validation côté client (Angular)
 - └ POST /participants ou /jury-members
 - └ Email de confirmation envoyé
 - └ Compteurs mis à jour
2. VÉRIFICATION DES CONDITIONS

- └ Dashboard vérifie toutes les 10 secondes
- └ GET /participants/count
- └ GET /jury-members/count
- └ GET /periods pour vérifier délai
- └ Si OK → Création de période

3. CRÉATION DE PÉRIODE

- └ POST /periods (mercredi 14h30 - vendredi 14h30)
- └ Statut → EN_PREPARATION
- └ Compte à rebours jusqu'au début

4. DÉMARRAGE AUTOMATIQUE

- └ À la date de début
- └ POST /hackathons/generate-teams
- └ Génération d'équipes équilibrées
- └ Sélection aléatoire d'un jury
- └ Statut → EN_COURS
- └ Affichage des équipes et sujet

5. EN COURS

- └ Compte à rebours jusqu'à la fin
- └ Affichage des informations
- └ Surveillance du timer

6. FIN

- └ À la date de fin
- └ Statut → TERMINE
- └ Planification du nettoyage (minuit)

7. NETTOYAGE AUTOMATIQUE

- └ À minuit après la fin
- └ DELETE /hackathons
- └ DELETE /teams
- └ DELETE /participants
- └ DELETE /jury-members
- └ Statut → EN_ATTENTE
- └ Prêt pour un nouveau cycle

Communication HTTP

Frontend (Angular)

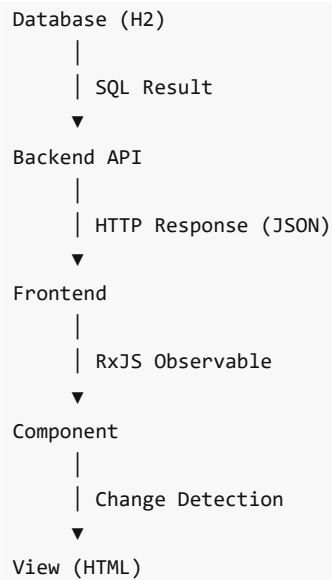
- |
- | HTTP Request (GET/POST/DELETE)



Backend API (Spring Boot)

- |
- | @RestController
- | Service Layer
- |
- | JPA Repository





Gestion des états

Approche réactive avec RxJS

L'application utilise RxJS pour la gestion réactive des états :

```
// Service avec état partagé
export class StatusService {
  private statusSubject = new BehaviorSubject<HackathonStatus>('EN_ATTENTE');
  public status$ = this.statusSubject.asObservable();

  updateStatus(status: HackathonStatus): void {
    this.statusSubject.next(status);
  }
}

// Composant qui écoute les changements
export class DashboardPage implements OnInit {
  ngOnInit(): void {
    this.statusService.status$.subscribe((status) => {
      this.currentStatus = status;
      this.cdr.detectChanges();
    });
  }
}
```

Lifecycle hooks

```
export class DashboardPage implements OnInit, OnDestroy {
  private destroy$ = new Subject<void>();

  ngOnInit(): void {
```

```

    // Initialisation et souscriptions
    this.loadData();
    this.startPolling();
  }

  ngOnDestroy(): void {
    // Nettoyage des souscriptions
    this.destroy$.next();
    this.destroy$.complete();

    // Nettoyage des timers
    if (this.pollingSubscription) {
      this.pollingSubscription.unsubscribe();
    }
  }
}

```

Tests

Configuration Vitest

Fichier: [tsconfig.spec.json](#)

```

{
  "extends": "../tsconfig.json",
  "compilerOptions": {
    "types": ["vitest"]
  }
}

```

Tests unitaires

Exemple de test pour DashboardPage

```

import { describe, it, expect, beforeEach } from 'vitest';

describe('DashboardPage', () => {
  let component: DashboardPage;

  beforeEach(() => {
    // Setup
    component = new DashboardPage(mockStatusService, mockHackathonService,
mockParticipantService);
  });

  it('should create', () => {
    expect(component).toBeTruthy();
  });

  it('should load participants on init', () => {

```

```
component.ngOnInit();
expect(component.participants.length).toBeGreaterThan(0);
});

it('should transition to EN_PREPARATION when conditions are met', () => {
  component.participantCount = 4;
  component.juryCount = 1;
  component.checkConditionsAndCreatePeriod();
  expect(component.currentStatus).toBe('EN_PREPARATION');
});
});
```

Commandes de test

```
# Exécuter tous les tests
npm test

# Exécuter les tests en mode watch
npm test -- --watch

# Exécuter les tests avec couverture
npm test -- --coverage
```

Déploiement

Build de production

```
# Build avec optimisations
npm run build

# Les fichiers sont générés dans dist/
```

Configuration Vercel

Fichier: [vercel.json](#)

```
{
  "rewrites": [
    {
      "source": "/(.*)",
      "destination": "/index.html"
    }
  ]
}
```

Déploiement sur Vercel

```
# Installation de Vercel CLI
npm i -g vercel

# Déploiement
vercel --prod
```

Configuration Angular pour production

Fichier: [angular.json](#)

```
{
  "configurations": {
    "production": {
      "fileReplacements": [
        {
          "replace": "src/app/env/env.dev.ts",
          "with": "src/app/env/env.prd.ts"
        }
      ],
      "optimization": true,
      "outputHashing": "all",
      "sourceMap": false,
      "namedChunks": false,
      "extractLicenses": true,
      "budgets": [
        {
          "type": "initial",
          "maximumWarning": "500kB",
          "maximumError": "1MB"
        }
      ]
    }
  }
}
```

Bonnes pratiques

Code quality

1. Type safety

```
// ✅ Bon
function getParticipant(id: number): Observable<Participant> {
  return this.http.get<Participant>(`/participants/${id}`);
}

// ❌ Mauvais
function getParticipant(id): any {
```

```

    return this.http.get(`/participants/${id}`);
}

```

2. Gestion des erreurs

```

// ✅ Bon
this.http
  .get<Participant[]>('/participants')
  .pipe(
    catchError((error) => {
      console.error('Error loading participants:', error);
      return of([]);
    })
  )
  .subscribe((participants) => {
    this.participants = participants;
  });

```

3. Nettoyage des souscriptions

```

// ✅ Bon - avec takeUntil
private destroy$ = new Subject<void>();

ngOnInit() {
  this.service.data$
    .pipe(takeUntil(this.destroy$))
    .subscribe(data => this.data = data);
}

ngOnDestroy() {
  this.destroy$.next();
  this.destroy$.complete();
}

```

4. Immutabilité

```

// ✅ Bon
const newArray = [...oldArray, newItem];

// ❌ Mauvais
oldArray.push(newItem);

```

Performance

1. Change Detection

```

// Utiliser ChangeDetectorRef pour les mises à jour manuelles
constructor(private cdr: ChangeDetectorRef) {}

updateData(): void {

```

```
this.data = newData;
this.cdr.detectChanges();
}
```

2. Lazy loading

```
// Routes avec lazy loading
{
  path: 'dashboard',
  loadChildren: () => import('./features/dashboard-page/dashboard-page').then(m =>
m.DashboardPage)
}
```

Sécurité

1. Validation des entrées

```
// Toujours valider côté client ET côté serveur
this.form = this.fb.group({
  email: ['', [Validators.required, Validators.email]],
  name: ['', [Validators.required, Validators.minLength(2)]],
});
```

2. Sanitization

```
// Angular sanitize automatiquement le HTML
// Mais attention aux innerHTML
```

Dépannage

Problèmes courants

L'application ne démarre pas

```
# Vérifier la version de Node
node --version # Doit être >= 20

# Supprimer node_modules et réinstaller
rm -rf node_modules package-lock.json
npm install
```

Le compte à rebours ne se met pas à jour

- Vérifier que `ChangeDetectorRef.detectChanges()` est appelé
- Vérifier les souscriptions dans `ngOnDestroy`
- Vérifier les timers avec `clearInterval`

Les transitions de statut ne fonctionnent pas

- Vérifier les dates dans la base de données

- Vérifier les constantes dans `dashboard.constants.ts`
- Vérifier les logs de la console
- Vérifier la connexion à l'API

Les équipes ne se génèrent pas

- Vérifier le nombre de participants (minimum 4)
- Vérifier le nombre de membres du jury (minimum 1)
- Vérifier les logs de l'API backend
- Vérifier que le hackathon est bien créé

Erreurs CORS

```
// Côté backend Spring Boot, vérifier la configuration CORS
@CrossOrigin(origins = ["http://localhost:4200"])
```

Logs et debugging

```
// Activer les logs détaillés
if (!environment.production) {
  console.log('Debug info:', data);
}

// Utiliser rxjs tap pour debugger les observables
this.service
  .getData()
  .pipe(
    tap((data) => console.log('Data received:', data)),
    catchError((error) => {
      console.error('Error:', error);
      return of(null);
    })
  )
  .subscribe();
```

Backend API (Spring Boot)

Vue d'ensemble

L'API backend est développée avec **Spring Boot 4.0.0** et fournit une API REST pour gérer toutes les opérations du hackathon.

Stack technique:

Technologie	Version	Usage
Spring Boot	4.0.0	Framework backend
Spring Data JPA	-	ORM et accès aux données
H2 Database	-	Base de données en mémoire

Lombok	-	Réduction du boilerplate
Java	17	Langage de programmation
Maven	-	Gestion des dépendances

Architecture Backend

```
src/main/java/com/kcode/hn_innov_api/
├── controller/           # Contrôleurs REST
│   ├── HackathonController.java
│   ├── ParticipantController.java
│   ├── JuryMemberController.java
│   ├── TeamController.java
│   └── EmailController.java
│
├── service/             # Interfaces de services
│   ├── HackathonService.java
│   ├── ParticipantService.java
│   ├── TeamService.java
│   └── ...
│
├── service/impl/        # Implémentations des services
│   ├── HackathonServiceImpl.java
│   ├── TeamServiceImpl.java
│   └── ...
│
├── entity/              # Entités JPA
│   ├── HackathonEntity.java
│   ├── ParticipantEntity.java
│   ├── TeamEntity.java
│   ├── PeriodEntity.java
│   ├── JuryMemberEntity.java
│   ├── StatusEntity.java
│   └── StatusEnum.java
│
├── repository/          # Repositories Spring Data
│   ├── HackathonRepository.java
│   ├── ParticipantRepository.java
│   └── ...
│
└── utils/               # Classes utilitaires
    ├── TechTeamNameGenerator.java
    ├── IcsGenerator.java
    └── DateUtil.java
```

Entités principales

ParticipantEntity

```

@Entity
@Table(name = "participants")
public class ParticipantEntity {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String firstName;
    private String lastName;
    private String email;
    private String skill;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "team_id")
    private TeamEntity team;
}

```

Contrôleurs REST

HackathonController

```

@RestController
@RequestMapping("/hackathons")
@CrossOrigin(origins = "*")
public class HackathonController {
    @PostMapping("/generate")
    private HackathonEntity createHackathon() {
        return service.create();
    }

    @DeleteMapping("/delete")
    private void deleteAll() {
        service.deleteAll();
    }
}

```

Services clés

- **HackathonService** : Génération de hackathons et d'équipes
- **TeamService** : Gestion des équipes et attribution des membres
- **ParticipantService** : Gestion des inscriptions de participants
- **StatusService** : Gestion des transitions d'états
- **EmailService** : Envoi de notifications par email

Configuration H2 Database

Base de données en mémoire H2 configurée dans `application.properties` :

- Persistance en mémoire pour les tests et développement
- Réinitialisation automatique au redémarrage

Utilitaires

- **TechTeamNameGenerator** : Génération de noms d'équipes créatifs
- **IcsGenerator** : Génération de fichiers .ics pour calendrier
- **DateUtil** : Manipulation et formatage des dates

Déploiement Backend

Fly.io : L'API est déployée sur <https://hn-innov-api.fly.dev>

Configuration via `fly.toml` et `Dockerfile` pour le déploiement automatique.

Annexes

Constantes de configuration

Fichier: <src/app/constantes/dashboard.constants.ts>

```
export const MIN_PARTICIPANTS = 4;
export const MIN_JURY_MEMBERS = 1;
export const MIN_DAYS_BETWEEN_PERIODS = 6;
export const TEST_START_OFFSET_MINUTES = 5;
export const TEST_END_OFFSET_MINUTES = 10;
```

Endpoints API

Méthode	Endpoint	Description
GET	/participants	Liste des participants
POST	/participants	Créer un participant
GET	/participants/count	Nombre de participants
DELETE	/participants	Supprimer tous les participants
GET	/jury-members	Liste des membres du jury
POST	/jury-members	Créer un membre du jury
GET	/jury-members/count	Nombre de membres du jury
DELETE	/jury-members	Supprimer tous les membres du jury
GET	/teams	Liste des équipes
POST	/teams	Créer une équipe
DELETE	/teams	Supprimer toutes les équipes
GET	/status	Statut actuel
PUT	/status	Mettre à jour le statut
GET	/periods	Liste des périodes
POST	/periods	Créer une période

GET	/hackathons	Liste des hackathons
POST	/hackathons	Créer un hackathon
POST	/hackathons/generate-teams	Générer les équipes
DELETE	/hackathons	Supprimer tous les hackathons

Ressources utiles

- [Documentation Angular](#)
- [Documentation RxJS](#)
- [Documentation Tailwind CSS](#)
- [Documentation TypeScript](#)
- [Vitest Documentation](#)

Dernière mise à jour: Janvier 2026

Auteur: Équipe de développement HackathoN Innov

Contact: [Voir page contact]